

Node.js Application Deployment on AWS EC2 using Ansible

1. Introduction

This project demonstrates how to deploy a Node.js web application on an AWS EC2 Ubuntu server using Ansible. The entire setup is automated using Ansible roles and playbooks, ensuring repeatable and reliable deployment. The project also involves configuring AWS networking components such as VPC, Subnet, Route Table, Network ACL, and Security Groups.

2. Objectives

- Deploy a Node.js application on AWS EC2
- Automate the deployment using Ansible
- Configure AWS networking for internet access
- Run the application and access it via public IP
- Follow industry best practices

3. Tools & Technologies Used

- AWS EC2 (Ubuntu 24.04)
- Ansible
- Node.js (v16)
- Git & GitHub
- Linux (Ubuntu)
- SSH

4. Architecture Overview

- Local Machine (Ansible Controller)
- AWS EC2 Instance (Managed Node)
- Public Subnet with Internet Gateway
- Security Group allowing SSH (22) and App Port (3000)

5. AWS Setup

5.1 EC2 Instance Configuration

- AMI: Ubuntu 24.04 LTS
- Instance Type: t2.micro
- Key Pair: node.pem
- Public IP Enabled

5.2 Networking Configuration

- VPC with Public Subnet
- Route Table: 0.0.0.0/0 → Internet Gateway
- Network ACL: Allow All Inbound & Outbound
- Security Group Inbound Rules:
 - SSH (22) – My IP
 - Custom TCP (3000) – 0.0.0.0/0

6. Ansible Project Structure

```
nodejs-ansible/  
├── ansible.cfg  
├── inventory  
├── playbook.yml  
├── roles/  
│   ├── nodejs/  
│   │   ├── tasks/  
│   │   └── main.yml
```

7. Ansible Configuration Files

7.1 ansible.cfg

```
[defaults]  
inventory = inventory  
host_key_checking = False  
remote_user = ubuntu  
private_key_file = /home/admin/node.pem  
interpreter_python = auto_silent
```

7.2 inventory

```
[nodejs]  
ec2 ansible_host=52.66.211.139 ansible_user=ubuntu  
ansible_ssh_private_key_file=/home/admin/node.pem
```

7.3 playbook.yml

```
---  
- name: Deploy Node.js Application on AWS EC2  
  hosts: nodejs  
  become: yes  
  
  roles:  
  - nodejs
```

8. Node.js Role Tasks

```
---
- name: Update apt cache
  apt:
    update_cache: yes
    cache_valid_time: 3600

- name: Install required packages
  apt:
    name:
      - curl
      - git
    state: present

- name: Add Node.js repository
  shell: curl -fsSL https://deb.nodesource.com/setup_16.x | bash -
  args:
    executable: /bin/bash

- name: Install Node.js
  apt:
    name: nodejs
    state: present

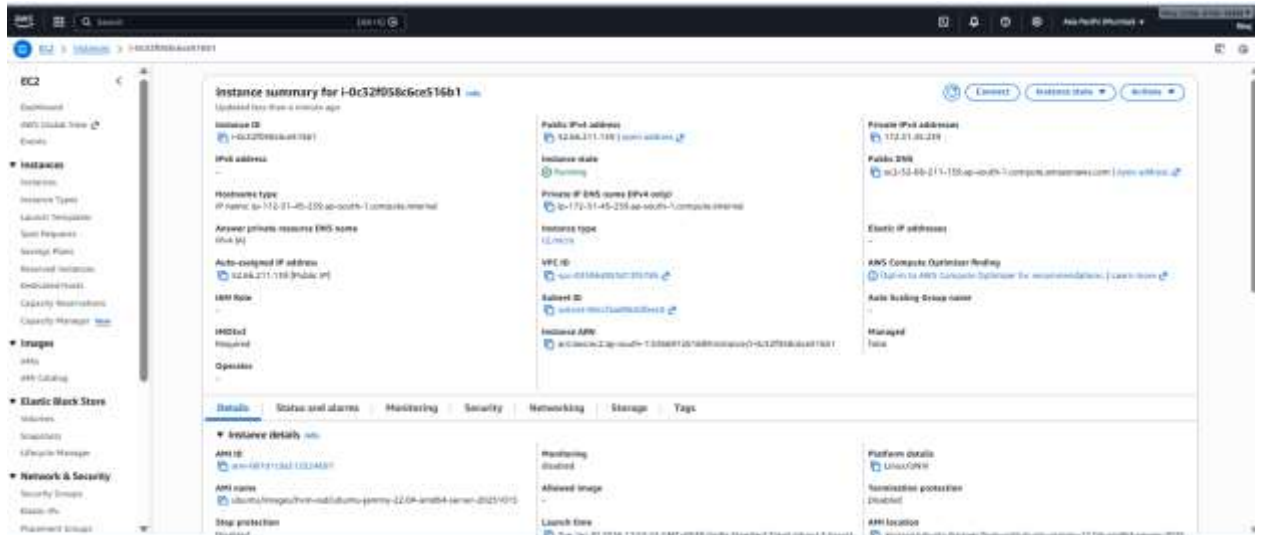
- name: Clone Node.js application
  git:
    repo: https://github.com/heroku/node-js-getting-started.git
    dest: /home/ubuntu/node-app
    force: yes

- name: Install npm dependencies
  npm:
    path: /home/ubuntu/node-app

- name: Start Node.js application
  shell: nohup npm start > app.log 2>&1 &
  args:
    chdir: /home/ubuntu/node-app
  environment:
    PORT: 3000
```

9. Execution Steps

1. Configure AWS EC2 and networking
 - Launch Ubuntu EC2 instance
 - Configure VPC, Subnet, Route Table, NACL, and Security Group
 - Enable ports 22 and 3000



2. Install Ansible on local machine
3. Create Ansible project structure

```
admin@RitejMachine:~/nodejs-ansible$ ls -R
.:
ansible.cfg  inventory  playbook.yml  roles

./roles:
nodejs

./roles/nodejs:
tasks

./roles/nodejs/tasks:
main.yml
admin@RitejMachine:~/nodejs-ansible$ |
```

4. Configure inventory and playbook
 5. Run Ansible playbook
- ansible-playbook playbook.yml

```

admin@RitejMachine:~/nodejs-ansible$ ansible-playbook playbook.yml

PLAY [Deploy Node.js Application on AWS EC2] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'ec2' is using the discovered Python interpreter at '/usr/bin/python3.10', but future installation of another Python in
terpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.19/reference_appendices/int
erpreter_discovery.html for more information.
ok: [ec2]

TASK [nodejs : Update apt cache] *****
ok: [ec2]

TASK [nodejs : Install required packages] *****
ok: [ec2]

TASK [nodejs : Add Node.js 16 repository] *****
changed: [ec2]

TASK [nodejs : Install Node.js] *****
ok: [ec2]

TASK [nodejs : Clone professional Node.js app] *****
ok: [ec2]

TASK [nodejs : Install npm dependencies] *****
ok: [ec2]

TASK [nodejs : Start Node.js application] *****
changed: [ec2]

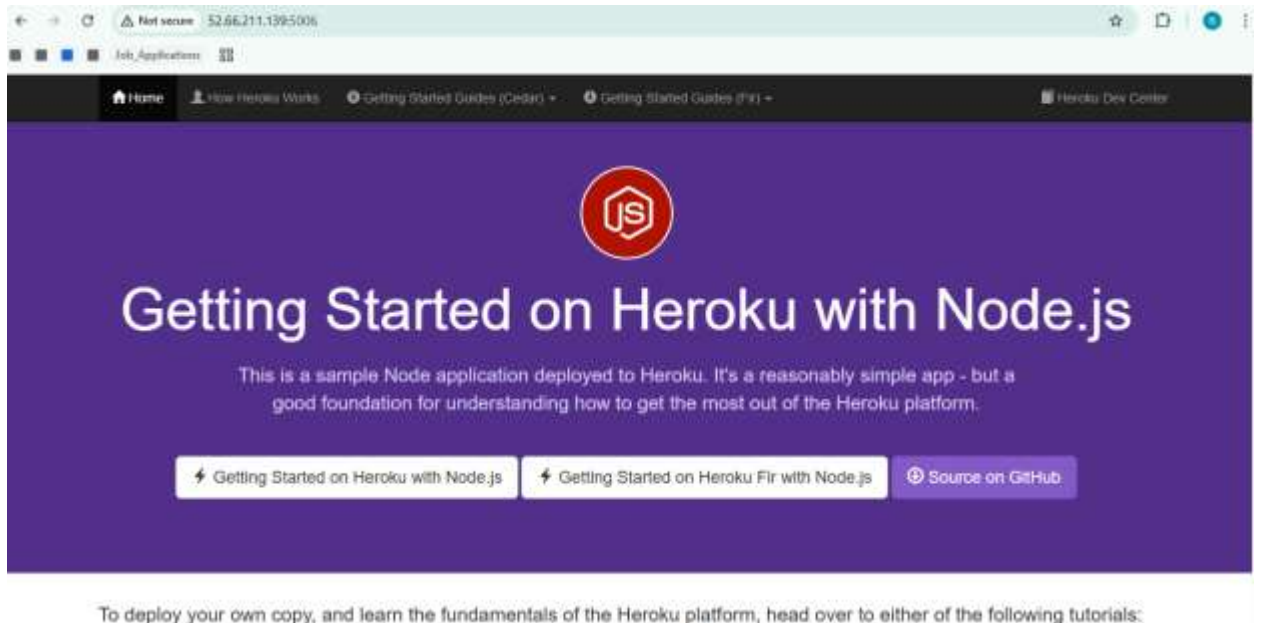
PLAY RECAP *****
ec2                : ok=8    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

admin@RitejMachine:~/nodejs-ansible$

```

10. Verification

- Access application in browser:



Browser showing Node.js application running on `http://52.66.211.139:5006`

11. Issues Faced & Resolutions

Issue	Resolution
No internet access	Fixed NACL outbound rules
SSH permission denied	Correct user and key pair
App not reachable	Identified correct port
File permission issue	Fixed ownership / used env variables

12. Best Practices Followed

- Infrastructure automation using Ansible
- Use of roles for modular design
- Environment variables for configuration
- Secure SSH access

13. Conclusion

This project successfully demonstrates automated deployment of a Node.js application on AWS EC2 using Ansible. It covers real-world DevOps practices including automation, cloud networking, and troubleshooting, making it suitable for academic submissions and interviews.